Predicting Credit Approval with Statistical Learning Methods

Math 448 Final Project

Nicholas Kane

December 2025

1 Executive Summary

An individual's financial success in the modern world is directly tied to their creditworthiness and credit score. Banks and lenders are often quiet about their methods of accepting and credit ranking, leaving many in the dark about why they were not approved for a credit card or credit line. The purpose of this project is to take data on 1389 individuals approved and not approved for a credit card, and use various predictors to see if there are any important features, or if an accurate model can be created. This data was published in Greene's 1992 text *Econometric Analysis, 5th Edition*, and contains data with 12 variables.

Several models were tested, including the following: Logistic Regression, Linear Disciminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), k-Nearest Neighbors (KNN), Classification Trees, Random Forests (RF), Boosting, Ridge Regression, and Lasso Regression. Most models returned decent results, but a few performed great. Logistic regression had the largest imbalance, having a low accuracy and a high AUC, whereas KNN and RF balanced their accuracy and AUC.

Most models included number of derogatory reports, income, number of active credit lines, and whether an individual owns their home, but reports and income stood above the rest in terms of importance. This aligns with standard thinking, and did not uncover any hidden patterns for acceptance.

This data was from 1992, and the culture around credit has changed in the past 30 years. Due to this, the first extension I would do is to look at a more modern data set. I also may check out various card companies (Visa, Mastercard, Discover, etc.) and see if they weigh factors differently.

2 Introduction

A high credit score is a major factor in modern economic success. There are many ways to build credit, but the easiest way for an average person is to obtain a credit card. Millions apply every year, but a fraction are accepted. Although there are more lenient cards and lenders, these may be predatory. Each lending service or bank has their own hidden list of factors that define an acceptable applicant. My goal with this project is to see if I can use prediction-based statistical learning methods to determine which factors affect credit card approval, as well as which have higher weights.

3 Data Cleaning

The data set for this project was compiled in 1992 by Greene and used in his textbook Econometric Analysis, 5th edition. The dataset was obtained from https://vincentarelbundock.github.io/Rdatasets/datasets.html The data contains 13 variables for 1389 applicants. The goal is to predict, based on the measured variables, whether an applicant will be accepted for a credit card or loan. A table containing the variables and what they represent can be found below.

Variable	**Description**
card	(Factor). Was the application for a credit card accepted?
reports	Number of major derogatory reports.
age	Age in years plus twelfths of a year.
income	Yearly income (in USD 10,000).
share	Ratio of monthly credit card expenditure to yearly income.
expenditure	Average monthly credit card expenditure.
owner	(Factor). Does the individual own their home?
selfemp	(Factor). Is the individual self-employed?
dependents	Number of dependents.
months	Months living at current address.
majorcards	Number of major credit cards held.
active	Number of active credit accounts.

Table 1: Variables for Credit Card Dataset

When importing the data, the first thing I checked was for empty or extra variables. There were no missing entries, however the first column of data is simply an index from 1-1389, so it was removed. I then used glimpse, as well as checked the summary of the data to make sure each data type makes sense for my project. The variables *card, owner* and *selfemp* were all factors and entered as yes/no, so I changed these to 1 and 0 for ease of analysis. Additionally, there were a few ages that were smaller than 1. Out of the 1389 data points, this was only true for about 6, a very small fraction of the sample, so I removed them. This is most likely due to entry error, since the age comes in years plus twelfths of a year. I also rounded some values so they are cleaner to work with, and collapse into nicer bins. I rounded age to the year, income and expenditure to two digits after the decimal, and share to six digits after the decimal.

After, I checked the balance of acceptances. It turns out that 77% of the data (1023) were accepted and 22% of the data (296) were not accepted. To address this, I used the ROSE package and undersampled the data to ensure an even spread of accepted/not accepted values for the dataset. The data set now contains 593 rows. I am aware that this may lead to better/worse performance during the validation step, but it is best to do for classification problems, especially logistic, which is a model I will be using.

4 Data Visualization

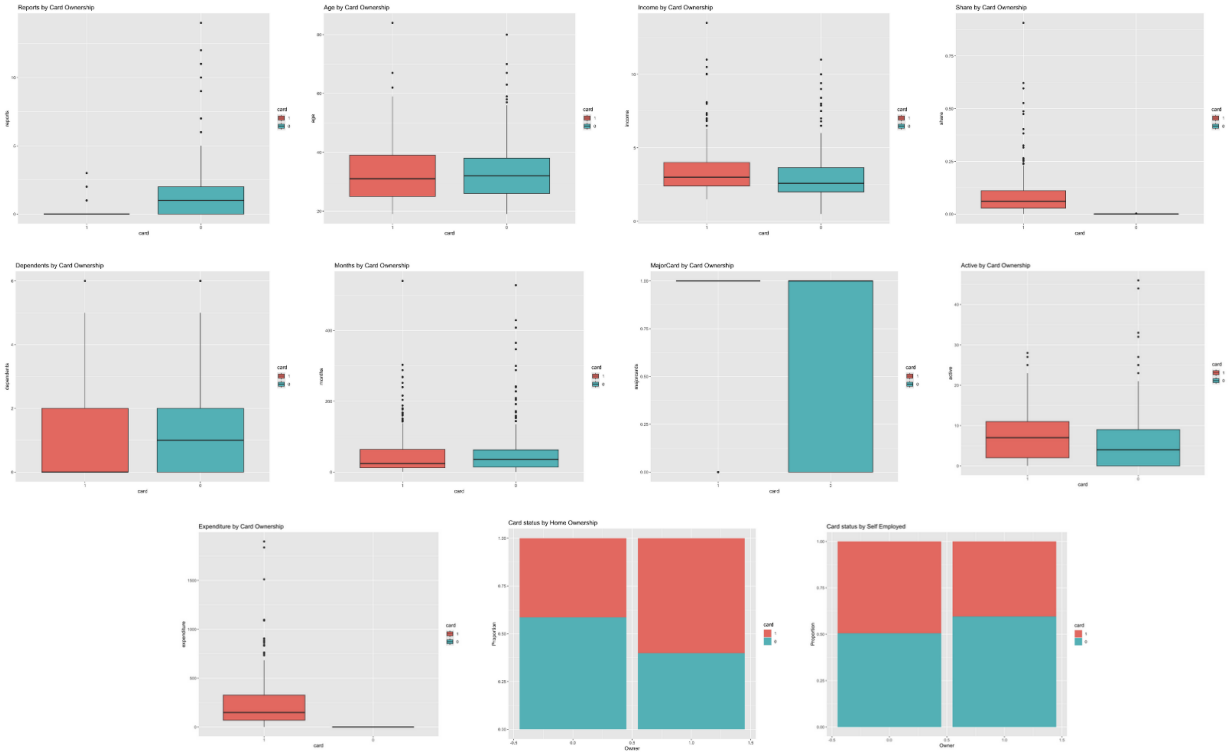

Figure 1: Box Plots for Predictors

The initial visualizations show that owner, income, reports, and active have the most influence. The rest are either uniform (age, dependents, months, selfemp), or have strange results (expenditure, share). Expenditure and share are 0 for all that were not accepted, since not having a card means there is no expenditure. If I included these, they would contribute to leakage, since these would not be available at the time of application. I removed these and the uniform variables.

5 Methods and Modeling

Before fitting any models, it is important to determine how these models will be evaluated and compared. I chose to use the accuracy (from a confusion matrix) as well as the AUC from the ROC curve. The accuracy is a good measure of how well any case is correctly classified, and the ROC/AUC is good for how the positives are ranked. To compute accuracy, I split the data into 80/20 training-testing split. I used set.seed(1) to ensure the same sets would be used throughout the analysis. Since the classes of the train/test sets may be imbalanced, accuracy will not be the best measure, but it is still a solid initial picture of the model fit.

5.1 Logistic Regression

The first model was logistic regression. This model was first tested on reports, income, active, and owner, however both logistic regression fit and backwards step selection dropped owner, so the final used logistic regression model was fit on reports, income, and active. When initially testing the accuracy, I set a cut-off point for prediction of 0.5, but found the accuracy was too low. I decreased it to 0.3, which weighs heavier towards sensitivity, and the model still had very low accuracy (0.328). However, the model had a very high AUC (0.843) and an overall good ROC curve. As observed in Figure 2, it is clear that logistic regression is much better at classifying positives, or whether someone will be accepted for a card. It is much worse at predicting negatives, or those not accepted. Overall, this model struggles with boundaries between classes.

Variable	Coefficients
Intercept	0.7791
reports	1.454
income	-0.253
active	-0.104

Table 2: Logistic Regression Coefficients

```
log.pred  0  1
       0  5 26
       1 52 33
```

Figure 2: Confusion Matrix for Logistic Regression

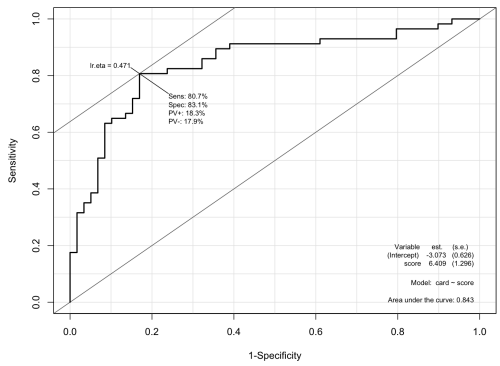

Figure 3: ROC for logistic

5.2 Linear Discriminant Analysis

The next model tested was linear discriminant analysis, which assumes equal covariance matrices between each class as well as predictors that follow a normal distribution. This raised the accuracy (0.767), however lowered the AUC (0.802). Overall, it has a better time with class boundaries and classification than logistic.

Variable	Coefficients
reports	0.539
owner	-0.188
income	-0.261
active	-0.063

Table 3: LDA Coefficients

```
          Actual
Predicted  0  1
        1 14 46
        0 43 13
```

Figure 4: Confusion Matrix for LDA

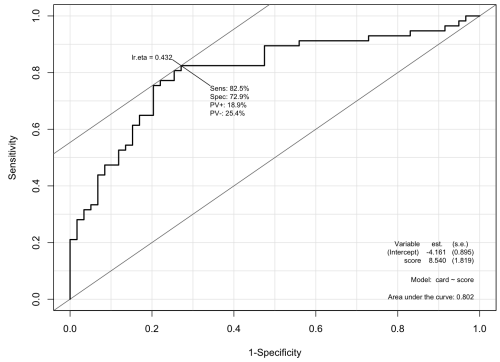

Figure 5: ROC for LDA

5.3 Quadratic Discriminant Analysis

QDA is similar to LDA, however it assumes each class has a different covariance matrix. Since the data is relatively small, QDA may not have enough information. Due to this we obtain both a smaller accuracy (0.672), and a higher AUC (0.823). Since QDA has non-linear boundaries, it has more variance, but classifies positives slightly better.

```
         Actual
Predicted  1  0
        1 55 34
        0  4 23
```

Figure 6: Confusion Matrix for QDA

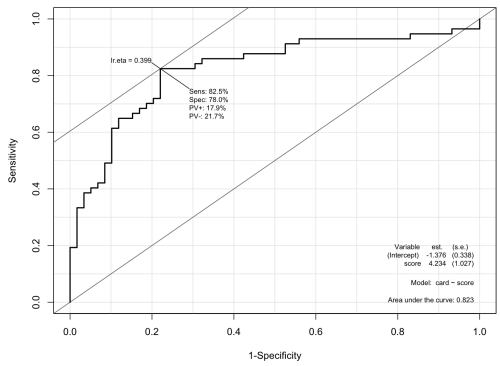

Figure 7: ROC for QDA

5.4 KNN

For KNN (k nearest neighbors) I used $k = 1, 3, 5$. Odd values are often best for KNN, since I am classifying, and there needs to be a majority. $k = 1$ overfits the noise, and $k = 5$ is too smooth, so the best option is $k = 3$, with an accuracy of 0.802. $k = 3$ obtains the best balance of bias-variance.

k	Accuracy
1	0.716
3	0.802
5	0.776

Table 4: KNN comparisons

5.5 Classification Trees

Classification trees had an overall lower accuracy (0.681) though a high AUC (0.812). This method worked much better with nonlinearities within the data. I also attempted to prune the tree, but this did not improve the accuracy enough for it to be worth it. The original model was already fairly simple only containing four predictors, so interpretability is not an issue. Trees are one of the easier methods to understand visually. From both trees it is clear that reports, and then active are the most important predictors.

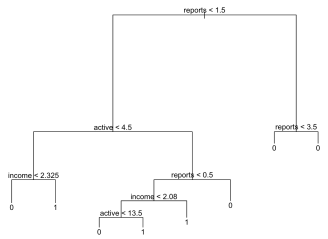

Figure 8: Classification Tree

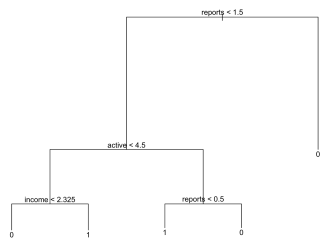

Figure 9: Pruned Tree

	Actual	
Predicted	0	1
1	24	46
0	33	13

Figure 10: Confusion Table for Tree

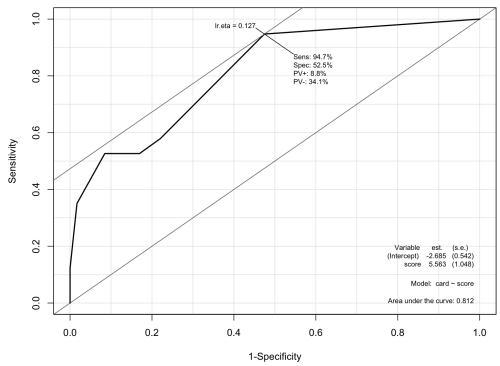

Figure 11: ROC for tree

5.6 Random Forests

Next, I modeled using random forests. From the variable importance plot in Figure 12, it is clear that income and reports are the two most important variables. Active is stronger for Gini than accuracy, but it is also important in the model. Owner is clearly the least important, which was eliminated in the logistic model. Interestingly, the random forests model assigns income more importance than active when compared to the classification trees. This may be due to the reduction of variance through bootstrapping. Random forests contain the highest accuracy (0.801) and the second highest AUC (0.842)

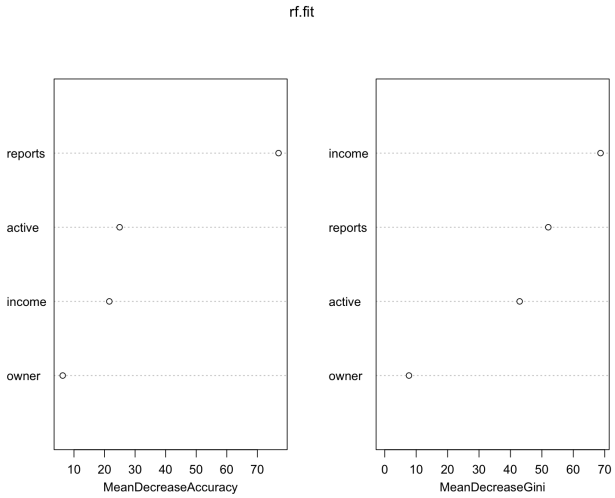

Figure 12: RF Variable Importance Plot

	Actual	
Predicted	0	1
1	14	50
0	43	9

Figure 13: Confusion table for RF

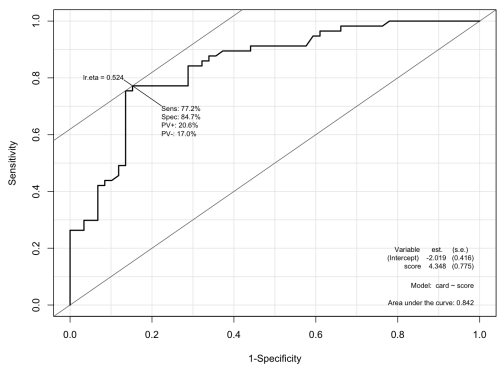

Figure 14: ROC for RF

5.7 Boosting

Next is boosting. This hones in on misclassified points and iteratively works to correct them. Boosting is much slower than other methods, and it therefore a very flexible model, which is why the accuracy is a bit lower (0.715), but the AUC is quite high (0.835). It also has an almost even split of misclassification: 41/17 zeroes and 42/16 ones.

```
          Actual
Predicted  0  1
        0 41 17
        1 16 42
```

Figure 15: Confusion matrix for Boosting

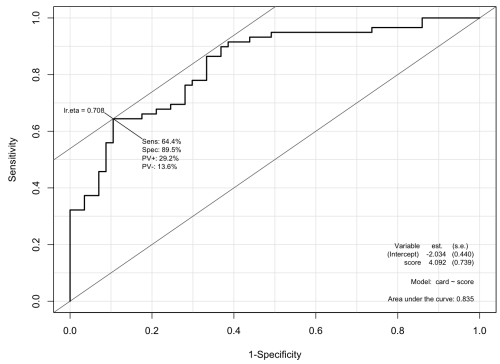

Figure 16: ROC for boosting

5.8 Ridge Regression

Ridge regression shrinks the parameters, but retains all variables, which lowers variance. Since our model was not too complex, there was no significant difference using ridge regression. The accuracy was moderate (0.759) and the AUC was high (0.828). Overall, this performed moderately compared to other models.

```
          Actual
Predicted  0  1
        0 39 10
        1 18 49
```

Figure 17: Confusion Matrix for Ridge Regression

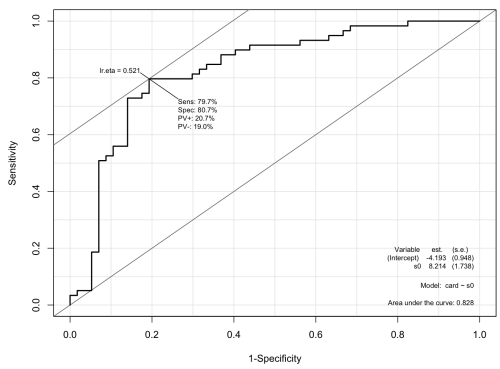

Figure 18: ROC for Ridge Regression

5.9 Lasso Regression

Lasso is similar to ridge, however it may zero out some of the predictors. It did not in my case, and in fact, has the same accuracy as ridge (0.759). However, it has a higher AUC (0.843), so I would pick lasso over ridge for these models.

```
          Actual
Predicted  0  1
        0 38  9
        1 19 50
```

Figure 19: Confusion Matrix for Lasso

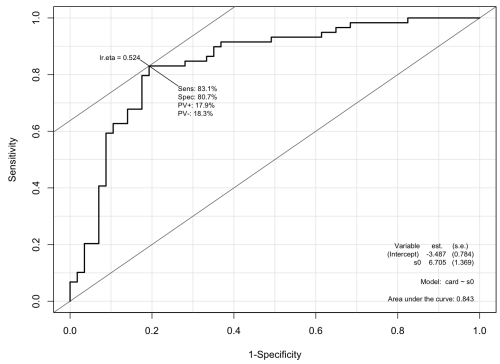

Figure 20: ROC for Lasso

6 Results and Comparison

As stated in the beginning of the modeling section, the two main comparisons I will be making here are the accuracy and AUC of the models. Ranking them by accuracy they are ordered as follows: KNN (3), RF, LDA, Ridge/LASSO, Boosting, Tree, QDA, Logistic. When ranking by AUC they are ordered: Logistic/LASSO, RF, KNN(3)/Boosting, Ridge, QDA, Tree, LDA. If choosing accuracy, KNN/RF would be the best, and if choosing ROC/AUC, then Logistic/LASSO/RF would be the best. Since it performs well in both situations, RF would be the best model overall. It balances both ranking and classification, and the logistic regression accuracy is far below random guessing.

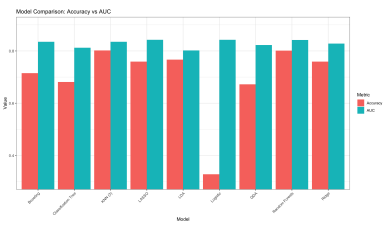

Figure 21: Model Comparison

Model	Accuracy	AUC
Logistic	0.328	0.843
LDA	0.767	0.802
QDA	0.672	0.823
KNN (3)	0.802	0.835
Classification Tree	0.681	0.812
Random Forests	0.801	0.842
Boosting	0.715	0.835
Ridge	0.759	0.828
LASSO	0.759	0.843

Table 5: Model Comparison

7 Conclusion and Future Direction

Through my project I found that when it comes to predicting credit card application status, derogatory reports and income are the two most important factors, followed by number of active credit accounts. This shows that consistently receiving derogatory credit reports heavily affects one's acceptance rate, which makes sense from the point of view of a lender. Although I began the project thinking that home ownership would affect the outcome, it turned out to be a small effect overall. The best model was random forests and KNN (3) was the second best overall. These models are both fairly easy to visualize and are both able to capture nonlinear boundaries as well as the interaction between terms. I found the low accuracy of the logistic regression surprising since every other method seemed to be around the same range of accuracy/AUC.

If I were to continue this project, the first step I would take is to resample the modeled data, to see if there are any significant differences in predictive models. I would also try to find a larger, and more recent data set, since credit and lending have increased in recent years. Lastly, I may model various card companies, like Visa, Mastercard, Discover, and American Express, and see if there are any differences in weights of variable importance for those companies.

8 Appendix: Code

```
    # ----- 448 project work ------
library(dplyr)
library(tidyr)
library(ROSE)
library(ggplot2)
library(boot)
library(MASS)
library(pROC)
library(Epi)
library(pROC)
library(lattice)
library(DAAG)
library(class)
library(tree)
library(randomForest)
library(glmnet)
library(gbm)

#----- load the data -----
credit_data <- read.csv("CreditCard.csv")

#-----inspect data/types-----
head(credit_data)
glimpse(credit_data)

#drop row names
credit_clean = credit_data[,-1]

summary(credit_clean)

#checking for N/A
colSums(is.na(credit_clean))

#Sorting
sort(credit_data$age)

credit_clean <- subset(credit_clean, age >= 18)

#Rounding some values
credit_clean$income <- round(credit_clean$income, digits = 2)
credit_clean$expenditure <- round(credit_clean$expenditure, digits = 2)
credit_clean$age <- round(credit_clean$age + .01)
credit_clean$share <- round(credit_clean$share, digits = 6)


#changing the categorical variables to 1 or 0
credit_clean <- credit_clean %>%
  mutate(
    card = ifelse(card == "yes", 1, 0),
    owner = ifelse(owner == "yes", 1, 0),
    selfemp = ifelse(selfemp == "yes", 1, 0)
  )

credit_clean$card <-as.factor(credit_clean$card)

#Checking summary, to make sure each data type is appropriate
summary(credit_clean)

#checking for balance
```

```
table(credit_clean$card)
prop.table(table(credit_clean$card))

#75% 1, 25% 0 so will take a smaller sample
set.seed(1)
credit_clean_under = ovun.sample(card ~ ., data=credit_clean, method="under")$data

table(credit_clean_under$card)
prop.table(table(credit_clean_under$card))

pairs(credit_clean_under)

#---- See relationship between predictors -----
#See relationship between reports and card
ggplot(credit_clean_under, aes(x = card, y = reports, fill = card)) +
  geom_boxplot() +
  labs(title = "Reports by Card Ownership")

#See relationship between age and card
ggplot(credit_clean_under, aes(x = card, y = age, fill = card)) +
  geom_boxplot() +
  labs(title = "Age by Card Ownership")

#See relationship between income and card
ggplot(credit_clean_under, aes(x = card, y = income, fill = card)) +
  geom_boxplot() +
  labs(title = "Income by Card Ownership")

#See relationship between share and card
ggplot(credit_clean_under, aes(x = card, y = share, fill = card)) +
  geom_boxplot() +
  labs(title = "Share by Card Ownership")

max(credit_clean_under$share[credit_clean_under$card == 0])


#See relationship between dependents and card
ggplot(credit_clean_under, aes(x = card, y = dependents, fill = card)) +
  geom_boxplot() +
  labs(title = "Dependents by Card Ownership")

#See relationship between months and card
ggplot(credit_clean_under, aes(x = card, y = months, fill = card)) +
  geom_boxplot() +
  labs(title = "Months by Card Ownership")

#See relationship between majorcard and card
ggplot(credit_clean_under, aes(x = card, y = majorcards, fill = card)) +
  geom_boxplot() +
  labs(title = "MajorCard by Card Ownership")

#See relationship between active and card
ggplot(credit_clean_under, aes(x = card, y = active, fill = card)) +
  geom_boxplot() +
  labs(title = "Active by Card Ownership")


#See relationship between expenditure and card
ggplot(credit_clean_under, aes(x = card, y = expenditure, fill = card)) +
  geom_boxplot() +
  labs(title = "Expenditure by Card Ownership")
```

```
# Check if any non-cardholders have nonzero expenditure
any(credit_clean_under$card == 0 & credit_clean_under$expenditure != 0)

#Remove Expenditure
credit_clean_under = credit_clean_under[,-6]

#plotting the home ownership status
ggplot(credit_clean_under, aes(x = owner, fill = card)) +
  geom_bar(position = "fill") +
  labs(title = "Card status by Home Ownership",
       x = "Owner", y = "Proportion")

#plotting the self employment status
ggplot(credit_clean_under, aes(x = selfemp, fill = card)) +
  geom_bar(position = "fill") +
  labs(title = "Card status by Self Employed",
       x = "Owner", y = "Proportion")


#strong
#reports, active, income, owner

#weak
# selfemp, months, age

#leakage
#expenditure, share, reports

#--- modeling stage ---

credit_modeling = credit_clean_under[, c(1,2,4,6,11)]

set.seed(12)

# --- create training sets ---
train_index <- sample(seq_len(nrow(credit_modeling)), size = 0.8 * nrow(credit_modeling))

train <- credit_modeling[train_index,]
test <- credit_modeling[-train_index,]



# --- initial log fit ---
log.fit1 <- glm(card ~ reports + income + owner + active,
                data = train, family = binomial

)

summary(log.fit1)

set.seed(1)
stepAIC(log.fit1, direction="backward")
stepAIC(log.fit1, direction="forward")

#backward step shows we can remove owner
log.fit2 <- glm(card ~ reports + income + active,
                data = train, family = binomial)

log.prob <- predict(log.fit2, test, type="response")
log.pred <- rep(0, length(test$card))
```

```
log.pred[log.prob > 0.3] = 1
table(log.pred, test$card)

log.accuracy <- mean(log.pred == test$card)
log.accuracy

log_dat <- data.frame(
  card = test$card, score = log.prob)

ROC(form = card ~ score, data = log_dat, plot = "ROC")

out0=cv.glm(data = credit_modeling, glmfit = log.fit2)
cost<-function(r,pi=0) {
  mean(abs(r-pi)>0.5)}
out1=cv.glm(data = credit_modeling, glmfit = log.fit2, cost, K=10)
cv.binary(log.fit2)
1-out0$delta[2] #loocv
1-out1$delta[2] #10-fold


# --- initial lda fit ---
lda.fit <- lda(card ~ reports + income + owner + active, data = train)
lda.pred <- predict(lda.fit, newdata = test)
confusion <- table(Predicted = lda.pred$class, Actual = test$card)
confusion
lda.accuracy <- mean(lda.pred$class == test$card)
lda.accuracy

# Extract predicted probabilities for the "yes" (or positive) class
prob.l <- lda.pred$posterior[, 2]

lda_dat <- data.frame(card = test$card, score = prob.l)

lda_roc <- ROC(form = card ~ score, data = lda_dat, plot = "ROC")

# --- initial qda fit ---
qda.fit <- qda(card ~ reports + income + owner + active, data = train)
qda.pred <- predict(qda.fit, newdata = test)
confusion <- table(Predicted = qda.pred$class, Actual = test$card)
confusion
qda.accuracy <- mean(qda.pred$class == test$card)
qda.accuracy

prob.q <- qda.pred$posterior[, 2]

qda_dat <- data.frame(card = test$card, score = prob.q)

lda_roc <- ROC(form = card ~ score, data = qda_dat, plot = "ROC")

# --- KNN ---
attach(credit_modeling)
vars <- c("reports", "income", "owner", "active")

train.means <- sapply(train[, vars], mean)
train.sds   <- sapply(train[, vars], sd)

train.X <- scale(train[, vars], center = train.means, scale = train.sds)
test.X  <- scale(test[, vars],  center = train.means, scale = train.sds)

train.card <- train$card
test.card  <- test$card
```

```
set.seed(1)

knn.pred=knn(train.X,test.X,train.card,k=1, prob = TRUE) #K=1
table(knn.pred,test.card)
knn1.accuracy <- mean(knn.pred == test.card)
knn1.accuracy

knn.pred=knn(train.X,test.X,train.card,k=3, prob = TRUE) #K=3
table(knn.pred,test.card)
knn3.accuracy  <- mean(knn.pred == test.card)
knn3.accuracy

knn.pred=knn(train.X,test.X,train.card,k=5, prob = TRUE) #K=5
table(knn.pred,test.card)
knn5.accuracy  <- mean(knn.pred == test.card)
knn5.accuracy

prob.raw <- attr(knn.pred, "prob")
pred.num <- as.numeric(knn.pred) - 1
prob.knn <- ifelse(pred.num == 1, prob.raw, 1 - prob.raw)

knn.dat <- data.frame(card = test.card, score = prob.knn)

ROC(form = card ~ score, data = knn.dat, plot = "ROC")

#K=3 is best

# --- Classification Tree ---
tree.fit <- tree(card ~ reports + income + owner + active, data = train)
plot(tree.fit)
text(tree.fit, pretty = 0)

tree.pred <- predict(tree.fit, newdata = test, type = "class")
confusion <- table(Predicted = tree.pred, Actual = test$card)
confusion

tree.accuracy <- mean(tree.pred == test$card)
tree.accuracy

tree.prob <- predict(tree.fit, newdata = test, type = "vector")[,2]

tree_dat <- data.frame(card = test$card, score = tree.prob)

ROC(form = card ~ score, data = tree_dat, plot="ROC")

cv.tree.fit <- cv.tree(tree.fit, FUN = prune.misclass)
plot(cv.tree.fit)
best.size <- cv.tree.fit$size[which.min(cv.tree.fit$dev)]

pruned.tree <- prune.misclass(tree.fit, best = best.size)

plot(pruned.tree)
text(pruned.tree, pretty = 0)

### --- Random Forest ---
train$card <- as.factor(train$card)
test$card  <- as.factor(test$card)
rf.fit <- randomForest(card ~ reports + income + owner + active, data = train,
                       ntree = 500, mtry = 2, importance = TRUE)

rf.fit
```

```
rf.pred <- predict(rf.fit, newdata = test, type = "class")
table(Predicted = rf.pred, Actual = test$card)

rf.accuracy <- mean(rf.pred == test$card)
rf.accuracy

rf.prob <- predict(rf.fit, newdata = test, type = "prob")[,2]
rf_dat <- data.frame(card = test$card, score = rf.prob)

ROC(form = card ~ score, data = rf_dat, plot = "ROC")

varImpPlot(rf.fit)

### --- Boosting ---
credit_modeling$card <- as.numeric(as.character(credit_modeling$card))
train$card <- as.numeric(as.character(train$card))
test$card <- as.numeric(as.character(test$card))

boost.fit <- gbm(formula = card ~ reports + income + owner + active, data = train, distribution = "bern
  interaction.depth = 3, shrinkage = 0.01, bag.fraction = 0.5, n.minobsinnode = 10, verbose = FALSE)

boost.prob <- predict(boost.fit, newdata = test, n.trees = 5000, type = "response")

boost.pred <- ifelse(boost.prob > 0.5, 1, 0)

confusion <- table(Predicted = boost.pred, Actual = test$card)
confusion

boost.accuracy <- mean(boost.pred == test$card)
boost.accuracy

boost_dat <- data.frame(card = test$card, score = boost.prob)

ROC(form = card ~ score, data = boost_dat, plot = "ROC")


# --- Ridge/Lasso
X.train <- model.matrix(card ~ reports + income + owner + active, data = train)[ , -1]
X.test  <- model.matrix(card ~ reports + income + owner + active, data = test)[ , -1]

y.train <- train$card
y.test  <- test$card

ridge.cv <- cv.glmnet(X.train, y.train, family = "binomial", alpha = 0)
plot(ridge.cv)

lambda.ridge <- ridge.cv$lambda.min
lambda.ridge
ridge.fit <- glmnet(X.train, y.train, family = "binomial", alpha = 0, lambda = lambda.ridge)
coef(ridge.fit)

ridge.prob <- predict(ridge.fit, newx = X.test, type = "response")
ridge.pred <- ifelse(ridge.prob > 0.5, 1, 0)

confusion <- table(Predicted = ridge.pred, Actual = y.test)
confusion
ridge.accuracy <- mean(ridge.pred == y.test)
ridge.accuracy

ridge_dat <- data.frame(card = y.test, score = ridge.prob)
```

```
ROC(form = card ~ s0, data = ridge_dat, plot = "ROC")


lasso.cv <- cv.glmnet(X.train, y.train, family = "binomial", alpha = 1)
lambda.lasso <- lasso.cv$lambda.min

lasso.fit <- glmnet(X.train, y.train, family = "binomial", alpha = 1, lambda = lambda.lasso)
coef(lasso.fit)

lasso.prob <- predict(lasso.fit, newx = X.test, type = "response")
lasso.pred <- ifelse(lasso.prob > 0.5, 1, 0)

confusion <- table(Predicted = lasso.pred, Actual = y.test)
confusion
lasso.accuracy <- mean(lasso.pred == y.test)
lasso.accuracy

lasso.dat <- data.frame(card = y.test, score = lasso.prob)
ROC(form = card ~ s0, data = lasso.dat, plot = "ROC")


#Comparison
results <- data.frame(
  Model = c("Logistic", "LDA", "QDA", "KNN (3)","Classification Tree",
            "Random Forests","Boosting", "Ridge", "LASSO"),
  Accuracy = c(0.328, 0.767, 0.672, 0.802, 0.681, 0.801, 0.715, 0.759, 0.759),
  AUC      = c(0.843, 0.802, 0.823, 0.835, 0.812, 0.842, 0.835, 0.828, 0.843))

results_long <- results |>
  pivot_longer(cols = c(Accuracy, AUC),
               names_to = "Metric",
               values_to = "Value")

ggplot(results_long, aes(x = Model, y = Value, fill = Metric)) +
  geom_col(position = "dodge") +
  coord_cartesian(ylim = c(0.3, 0.9)) +
  labs(title = "Model Comparison: Accuracy vs AUC",
       x = "Model", y = "Value") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```